

TECHNICAL DOCUMENTATION

VulnScan Toolkit

Modular Web Vulnerability Scanning Orchestrator

Version 0.1 (MVP) · July 2026 · CIESC / Innovation Project
Author: Higor Silva

1. Overview

VulnScan Toolkit is a containerized, modular orchestrator that automates web application vulnerability scanning by integrating established open-source security tools (sqlmap, OWASP ZAP, Nikto, Nuclei) behind a single command-line interface. It normalizes every tool's output into a common data model and produces a consolidated, professional report (HTML and PDF) containing an executive summary, CVSS-based criticality, technical detail, and mitigation guidance.

The goal is not to reinvent vulnerability scanning engines, but to remove the operational friction of running multiple disconnected tools by hand, reconciling inconsistent output formats, and manually assembling a report for stakeholders.

2. Architecture

The system is organized in four decoupled layers:

Layer	Responsibility	Location
CLI / Entry point	Argument parsing, authorization confirmation, run bootstrap	<code>src/main.py</code>
Core	Scope enforcement, orchestration, normalized <code>Finding</code> model	<code>src/core/</code>
Scan Modules	Adapters that invoke external tools and normalize their output	<code>src/modules/</code>
Report Engine	Aggregation, statistics, HTML/PDF rendering	<code>src/report/</code>

The core is intentionally decoupled from any specific scanning tool. Every module implements the same contract (`BaseScanModule.run() -> list[Finding]`), so the orchestrator and the report engine never need to know which underlying tool produced a result. Adding, removing, or replacing a tool only requires touching its module and its registry entry.

2.1 Data flow

1. User invokes the CLI with a target URL, scan profile, and an explicit authorization flag.
2. `ScopeGuard` checks the target against the allowlist in `config/scope.yaml`; execution is aborted if the host is not authorized.
3. `ScanOrchestrator` resolves the module list (from the profile or an explicit `--modules` override) and executes each module sequentially, capturing raw tool output for audit purposes.
4. Each module normalizes its findings into `Finding` objects (severity, CVSS vector/score, endpoint, parameter, evidence, remediation).
5. `ReportGenerator` aggregates all findings, computes summary statistics, and renders the Jinja2 template into HTML, optionally converting it to PDF via WeasyPrint.

3. Technology Stack

Component	Choice	Rationale
Language	Python 3.11+	Mature subprocess/parsing ecosystem; native language of most offensive tooling
Config	PyYAML	Human-editable scope/settings files
Templating	Jinja2	Clean separation between report logic and presentation
PDF rendering	WeasyPrint	Converts the same HTML/CSS to PDF without external binaries like wkhtmltopdf
Containerization	Docker (debian:bookworm-slim)	Isolates all offensive tooling dependencies from the host (e.g. Kali), avoiding library conflicts

4. Scanning Modules

Module	Vulnerability class	Underlying tool	Integration
recon	General misconfiguration / information disclosure	Nikto	CLI, JSON output
sqli	SQL Injection	sqlmap	CLI (<code>--batch</code>), stdout parsing
xss	Reflected / Stored / DOM XSS	OWASP ZAP (baseline scan)	CLI (<code>zap-baseline.py -J</code>), JSON alert parsing
xxe	XML External Entity	Nuclei (tag <code>xxe</code>)	CLI (<code>-jsonl</code>), line-delimited JSON parsing

Scan profiles bundle module sets for common use cases: `quick` runs recon + xss for a fast pass, `full` runs all four modules, and `custom` allows explicit module selection via `--modules`.

5. Normalized Finding Model

Every module returns a list of `Finding` objects (`src/core/finding.py`), the single contract between scan modules and the report engine:

```
module, title, description, severity (enum),
cvss_vector, cvss_score, endpoint, parameter,
evidence, recommendation, references[], tool_source
```

Severity uses a five-level enum (Critical, High, Medium, Low, Informational) mapped consistently regardless of the originating tool's own severity taxonomy (e.g. ZAP risk levels, Nuclei severity tags).

6. Report Engine

The report engine (`src/report/generator.py`) performs three steps: it computes aggregate statistics (finding counts per severity, average CVSS score, modules executed), sorts findings by criticality, and renders `report/templates/report.html.j2` . The resulting document contains four sections: Executive Summary, Criticality Overview (visual severity breakdown), Technical Detail (per-finding endpoint, parameter, evidence, CVSS), and Mitigation Recommendations. HTML is always produced; PDF is generated on request via WeasyPrint, using the exact same HTML/CSS so both formats stay visually consistent.

7. Security Controls

Scope enforcement is mandatory, not optional. ScopeGuard reads an allowlist of authorized hosts (`config/scope.yaml` , supporting wildcard patterns) and refuses to run any module against a target that is not listed. In addition, the CLI requires an explicit `--i-have-authorization` flag before any scan starts, as a deliberate friction point against accidental misuse.

All raw tool output (stdout/stderr/JSON) is preserved per run under `reports/<run_id>/<module>/` for traceability and audit purposes.

8. Deployment

The tool ships as a single Docker image built from the provided `Dockerfile` , installing sqlmap and Nikto via apt, Nuclei via `go install` , and the OWASP ZAP baseline script from the official release archive. `docker-compose.yml` mounts `./reports` and `./config` as volumes so scan output and configuration persist outside the container.

```
docker compose build
docker compose run --rm vulnscan \
  --target https://app-test.local \
  --profile full \
  --format both \
  --i-have-authorization
```

9. Known Limitations (MVP)

- Modules run sequentially; no parallelization yet (planned for a later sprint).
- sqlmap output is parsed from stdout via regex; a future iteration will consume sqlmap's structured output directly for robustness.
- No persistent scan history/database; each run is a standalone directory under `reports/` .
- No authenticated-scan support (login flows) in the current module set.

